

Modelo relacional de datos

Bases de datos



Indice

- 1. Introducción
- 2. Estructura
- 3. Restricciones
- 4. Transformación E-R a esquema relacional
- 5. Normalización
- 6. Álgebra relacional

1. Introducción

El modelo de datos relacional fue desarrollado por Codd. El modelo de Codd persigue al igual que la mayoría de los modelos de datos los siguientes objetivos:

- **Independencia física de los datos**, esto es, el modo de almacenamiento de los datos no debe influir en su manipulación lógica.
- **Independencia lógica de los datos**, es decir, los cambios que se realicen en los objetos de la base de datos no deben repercutir en los programas y usuarios que acceden a ella.
- **Flexibilidad**, para presentar a los usuarios los datos de la forma más adecuada.
- **Uniformidad**, en la presentación de la lógica de los datos, que son tablas, lo que facilita la manipulación de la base de datos por parte de los usuarios.
- **Sencillez**, este modelo es fácil de comprender y utilizar por el usuario.
- Para conseguir estos objetivos Codd introduce el concepto de **relación (tablas) como estructura básica** del modelo, todos los datos de una base de datos se representa en forma de relaciones cuyo contenido varía en el tiempo.

2. Estructura

La relación es el elemento básico del modelo relacional y se representa como una tabla, en la que se puede distinguir:

- El nombre de la **tabla**.
- El conjunto de columnas que representan las propiedades de la tabla y que se denominan **campos o atributos**.
- El conjunto de filas, llamadas **registros o tuplas** que contienen los valores que toman cada uno de los atributos para cada elemento de la relación.

Una relación tiene una serie de elementos característicos que la distinguen de una tabla:

- No admiten filas duplicadas.
- Las filas y las columnas no están ordenadas.
- La tabla es plana, es decir, en el cruce de una fila y una columna solo puede haber un valor

Tabla ALUMNOS

Num Mat	Nombre	Apellidos	Cod_Curso
1001	Juan	Cabello	1
1002	Dolores	García	1
1003	Jesús	Sánchez	2



Dominio

Se define **DOMINIO** como el conjunto finito de valores **homogéneos** (todos del mismo tipo) y **atómicos** (son indivisibles) que puede tomar cada atributo. Los valores contenidos en una columna pertenecen a un dominio que previamente se ha definido.

Todos los dominios tienen un nombre y un tipo de datos asociados. Existen dos tipos de dominios:

- **Generales:** Son aquellos cuyos valores están comprendidos entre un máximo y un mínimo. Por ejemplo, el Código_postal, que está formado por todos los números enteros positivos de 5 cifras.
- **Restringidos:** Dando sus posibles valores. Por ejemplo: días de la semana = {lunes, martes, miércoles, ...}.

Tipos de datos

- Los dominios se pueden definir con tipos de datos. Algunos ejemplos de tipos de datos posibles:
 - **Carácter:** Conjunto de caracteres alfanuméricos. Se puede poner el tamaño del campo entre paréntesis.
 - **Entero:** conjunto de números enteros ..., -2, -1, 0, 1, 2, ...
 - **Decimal:** conjunto de números con decimales. Ej. 2,41 -3,2
 - **Fecha:** Conjunto de valores de tipo fecha. Ej. 30/11/2015
 - **Hora:** Conjunto de datos de tipo hora. Ej. 20:50:15
 - **Booleano:** Tiene dos valores (verdadero ó falso , 1 ó 0)
 - **Memo:** Texto muy largo. Ej. Un artículo periodístico.
 - **Objeto:** Cualquier objeto binario. Ej. Archivo de imagen, de audio, etc.

Atributos

Se define **ATRIBUTO** como el papel o rol que desempeña un dominio en una relación (una característica). Representa el uso de un dominio para una determinada relación. El atributo aporta un significado semántico a un dominio. Por ejemplo, en la relación **ALUMNOS** podemos considerar los siguientes atributos y dominios:

- Atributo **NUM_MAT**. Dominio: conjunto de enteros formados por 4 dígitos.
- Atributo **NOMBRE**. Dominio: conjunto de 15 caracteres.
- Atributo **APELLIDOS**. Dominio: conjunto de 20 caracteres.
- Atributo **CURSO**. Dominio: conjunto de 7 caracteres.

Tablas

Las tablas se representan mediante una cuadrícula con filas y columnas. Un SGBD solo necesita que el usuario pueda percibir la BD como un conjunto de tablas.

En el modelo relacional las relaciones se utilizan para almacenar información sobre los objetos que se representan en la BD. Se representa como una tabla bidimensional en la que las filas corresponden a registros individuales y las columnas a los campos o atributos de esos registros. La relación está formada por:

- **Atributos (columnas o campos).** Se trata de cada una de las **columnas de la tabla**. Las columnas tienen un nombre y pueden guardar un conjunto de valores. Una columna se identifica siempre por su nombre, nunca por su posición. El orden de las columnas en una tabla es irrelevante.
- **Tuplas (filas o registros).** Cada tupla representa una fila de la tabla. En la siguiente tabla vemos que aparecen tres tuplas o filas, y cuatro atributos (num_mat, nombre, apellidos, curso).

Esquema e instancia de una tabla

- Un relación se compone del esquema y de la instancia.
- Si consideramos la representación tabular anterior, el esquema correspondería a la cabecera de la tabla y la instancia correspondería al cuerpo.

ALUMNOS

Num Mat	Nombre	Apellidos	Cod_Curso
1001	Juan	Cabello	1
1002	Dolores	García	1
1003	Jesús	Sánchez	2

} Esquema

} Instancia

Notaciones

Vamos a representar el esquema de una relación de dos maneras diferentes:

- Alumnos (Num_mat, Nombre, Apellidos, Cod_curso)



Alumnos
<u>Num_mat</u>
Nombre
Apellidos
Cod_curso

Tablas

De las tablas se derivan los siguientes conceptos:

- **Cardinalidad:** Es el número de filas de la tabla. En el ejemplo anterior, es TRES.
- **Grado:** Es el número de columnas de la tabla. En el ejemplo anterior, es CUATRO.
- **Valor:** Esta representado por la intersección entre una fila y columna. Por ejemplo, en la tabla anterior, son valores “CABELLO” “JUAN”, 1001, 1, ...
- **Valor nulo (null):** Representa la ausencia de información.

Las relaciones tienen las siguientes características:

- Cada relación tiene un nombre y este es distinto de los demás.
- Los valores de los atributos son atómicos: en cada tupla, cada atributo toma un solo valor.
- No hay dos atributos que se llamen igual.
- El orden de los atributos es irrelevante; no están ordenados.
- Cada tupla es distinta de las demás; no hay tuplas duplicadas.
- Al igual que los atributos, el orden de las tuplas es irrelevante; las tuplas no están ordenadas.

Claves

En una relación no hay tuplas repetidas; se identifica de un modo único mediante los valores de sus atributos. Toda fila debe estar asociada con una clave que permite identificarla. A veces las filas se pueden identificar con un único atributo, pero otras veces es necesario recurrir a más de un atributo.

La **clave candidata** de una relación es el conjunto de atributos que identifica de forma única y mínima cada tupla de la relación. Siempre hay una clave candidata.

- Una relación puede tener más de una clave candidata entre las cuales se distinguen:
- **Clave primaria o principal:** aquella clave candidata que el usuario escoge para identificar las tuplas de la relación. No puede tener valores nulos.
- **Clave alternativa:** aquellas claves candidatas que no han sido escogidas como clave primaria.
- La **Clave ajena** de una relación R2 es el conjunto de atributos cuyos valores han de coincidir con los valores de la clave primaria de otra relación R1.

Ejemplo tabla Alumnos

Para el ejemplo de la tabla Alumnos

- Clave candidata: Num_mat | DNI
- Clave primaria: Num_mat
- Clave alternativa: DNI
- Clave ajena: Curso

Ejemplo

Tabla Curso (R1)

PK: Cod_Curso, AK: Nombre

Cod_Curso	Nombre
1	1º ASIR
2	2º ASIR
3	1º BACH

Tabla Alumnos (R2)

PK: Num Mat FK: Cod_Curso

Num Mat	Nombre	Apellidos	Cod_Curso
1001	Juan	Cabello	1
1002	Dolores	García	1
1003	Jesús	Sánchez	2

Ejemplo

Tabla departamentos (R1)

PK: N° Depto , AK: Nombre

N° Depto	Nombre	Presupuesto
D1	Marketing	1000
D2	Desarrollo	1200
D3	Investigación	5000

Tabla empleados (R2)

PK: Num Emp FK: N° Depto

Num Emp	Apellidos	Salario	N° Depto
E1	López	500	D1
E2	Fernández	1000	D2
E3	García	1200	D3

3. Restricciones

En todos los modelos de datos existen restricciones que a la hora de diseñar una base de datos se han de tener en cuenta. Los datos almacenados en la base de datos han de adaptarse a las estructuras impuestas por el modelo y deben cumplir una serie de reglas para garantizar que son correctas. Los tipos de restricciones son:

- Restricciones inherentes al modelo
- Restricciones semánticas o de usuario
 - Restricción de Clave primaria (PRIMARY KEY)
 - Restricción de Unicidad (UNIQUE)
 - Restricción de Obligatoriedad (NOT NULL)
 - Restricción de clave ajena o integridad referencial (FOREIGN KEY)
 - Restricciones de verificación (CHECK)
 - Aserciones (ASSERTION)
 - Disparadores (TRIGGER)

Restricciones inherentes al modelo

Indican las características propias de una relación que ha de cumplirse obligatoriamente, y que diferencian una relación de una tabla:

- No hay dos tuplas iguales.
- El orden de las tuplas y los atributos no es relevante.
- Cada atributo sólo puede tomar un único valor del dominio al que pertenece.
- Ningún atributo que forme parte de la clave primaria de una relación puede tomar un valor nulo.

Restricciones semánticas o de usuario

Representan la semántica del mundo real. Éstas hacen que las ocurrencias de los esquemas de la base de datos sean válidos. Los mecanismos que proporcionan el modelo para este tipo de restricciones son los siguientes:

- ***Restricción de clave primaria (PRIMARY KEY)***, permite declarar uno o varios atributos como clave primaria de una relación.
- ***Restricción de unicidad (UNIQUE)***, permite definir claves alternativas. Los valores de los atributos no pueden repetirse.
- ***Restricción de obligatoriedad (NOT NULL)***, permite declarar si uno o varios atributos no pueden tomar valores nulos. (Por ejemplo: podríamos declarar que el atributo “Apellidos” de la tabla anterior “Empleados”, no pueda dejarse en blanco).
- **Restricción Clave ajena (FOREIGN KEY)**. Ver diap. siguiente.

Restricción clave ajena o Integridad referencial (FOREIGN KEY)

La **integridad referencial** indica que los valores de clave ajena en la relación hijo se corresponden con los de claves primarias en la relación padre. (Por ejemplo: que no pueda aparecer en la tabla “Ventas” el código de un producto si dicho producto no se encuentra en la tabla “Productos”).

Además de definir las claves ajenas hay que tener en cuenta las operaciones de borrado y actualización que se realizan sobre las tuplas de la relación referencial. Las posibilidades son las siguientes:

- Borrado y/o modificación en cascada. (CASCADE)
- Borrado y/o modificación restringido. (RESTRICT)
- Borrado y/o modificación con puesta a nulos (SET NULL)
- Borrado y/o modificación con puesta a valor por defecto (SET DEFAULT)

Notación para claves ajenas

- Notación1: Se expresa mediante una línea dirigida que va desde la clave ajena de la tabla hija hasta la clave primaria de la tabla padre.
- Notación 2: Se expresa mediante una línea que va desde la clave primaria de la tabla padre hasta la clave ajena de la tabla hija.

Opciones de borrado y/o modificación

Borrado y/o modificación en cascada. (Cascade). El borrado o modificación de una tupla en la relación padre (relación con la clave primaria) ocasiona un borrado o modificación de las tuplas relacionadas con la relación hija (relación que contiene la clave ajena). EJEMPLO: En el caso de empleados y departamentos, si se borra un departamento de la tabla TDEPART se borrarán los empleados que pertenecen a ese departamento. Igualmente ocurrirá si se modifica el NUMDEPT de la tabla TDEPART: esa modificación se arrastra a los empleados que pertenezcan a ese departamento.

Borrado y/o modificación restringido. (Restrict). En este caso no es posible realizar el borrado o la modificación de las tuplas de la relación padre si existen tuplas relacionadas en la relación hija. Es decir, no podría borrar un departamento que tiene empleados.

Borrado y/o modificación con puesta o nulos (Set null). Esta restricción permite poner la clave ajena en la tabla referenciada a NULL si se produce el borrado o modificación en la tabla primaria o padre. Así pues, si se borra un departamento, a los empleados de ese departamento se asignará NULL en el atributo NUMDEPT.

Borrado y/o modificación con puesta o valor por defecto (Set default). En este caso, el valor que se pone en la claves ajenas de la tabla referenciada es un valor por defecto que se habrá especificado en la creación de la tabla.

Notación opciones de borrado y/o modificación

- Las opciones de borrado y/o modificación se expresarán como *nombre de operación “dos puntos” nombre de opción*.
- Operación
 - “B” para borrado
 - “M” para modificación
- Opción
 - “C” para casada
 - “R” para restringido
 - “D” para puesta a valor por defecto
 - “N” para puesta a valor nulo

Operación:
Opción

Ejemplos:

B:N

M:C

B:C

M:C

Borrado en Cascada (Cascade)

Tabla departamentos

PK: N° Depto , AK: Nombre

1

N° Depto	Nombre	Presupuesto
D1	Marketing	1000
D2	Desarrollo	1200
D3	Investigación	5000

Tabla empleados

PK: Num Emp FK: N° Depto

2

Num Emp	Apellidos	Salario	N° Depto
E1	López	500	D1
E2	Fernández	1000	D1
E3	García	1200	D3

Borrado Restringido (Restrict)

Tabla departamentos

PK: N° Depto , AK: Nombre

N° Depto	Nombre	Presupuesto
D1	Marketing	1000
D2	Desarrollo	1200
D3	Investigación	5000

Tabla empleados

PK: Num Emp FK: N° Depto

Num Emp	Apellidos	Salario	N° Depto
E1	López	500	D1
E2	Fernández	1000	D1
E3	García	1200	D3

Borrado con puesta a nulos (SET NULL)

1

Nº Depto	Nombre	Presupuesto
D1	Marketing	1000
D2	Desarrollo	1200
D3	Investigación	5000

Tabla empleados

PK: Num Emp FK: Nº Depto

Num Emp	Apellidos	Salario	Nº Depto
E1	López	500	Null
E2	Fernández	1000	Null
E3	García	1200	D3

2

Restricciones semánticas o de usuario

- **Restricción de verificación (Check)** Esta restricción permite especificar condiciones que deben cumplir los valores de los atributos. Cada vez que se realice una inserción o una actualización de datos se comprueba si los valores cumplen la condición. Rechaza la operación si no se cumple.
- Ejemplo: que el campo PRECIO_VENTA sea mayor que 0.
- Ejemplo: Que el campo EDAD sea mayor que 0, y (por ejemplo) menor que 120.

Notación para restricción de verificación

- Se pondrá como una anotación en el campo que indique dicha restricción.

4. Transformación de un esquema E-R a esquema relacional

Una vez obtenido el esquema conceptual mediante el modelo E-R hay que definir el modelo lógico de datos. Las reglas básicas para transformar un esquema conceptual E-R a un esquema relacional son las siguientes:

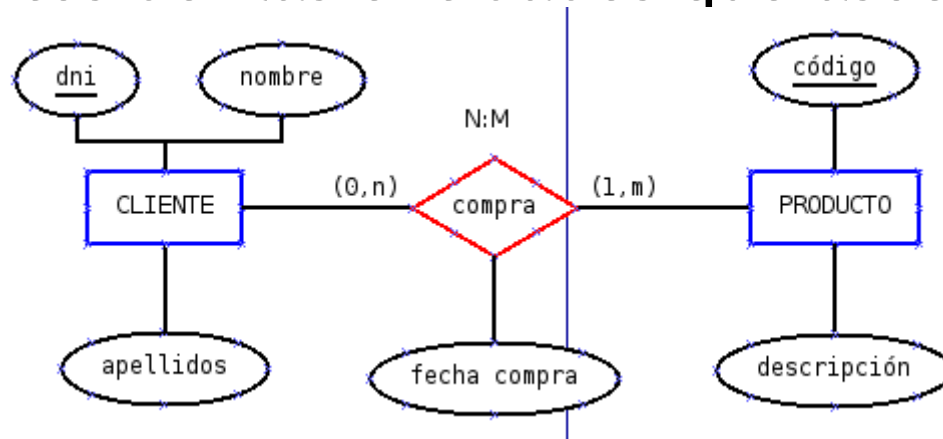
- Toda entidad se transforma en una tabla.
- Todo atributo se transforma en columnas dentro de una tabla.
- El identificador único de la entidad se convierte en clave primaria.

Transformación de atributos

Modelo E-R	Modelo relacional (Restricción)
Entidad	Tabla
Atributo Por defecto los atributos son simples, almacenados, monovalor y opcionales	Campo
AIP	Clave primaria PK
AIA	Clave alternativa AK (UNIQUE)
Atributo compuesto	Se crea un campo por cada atributo simple que forma parte de él
Atributo derivado	Se crea un campo con un trigger asociado que permita actualizar el campo
Atributo multivalor	Se crea una nueva tabla que tenga como clave primaria el atributo multivalor junto con la clave primaria de la tabla
Atributo obligatorio	Se crea un campo con restricción NOT NULL

Transformación de relaciones N:M

- Toda relación N:M se transforma en una tabla que tendrá como clave primaria la concatenación de los atributos de las entidades que asocia .



CLIENTE(dni, nombre, apellidos)

PRODUCTO(código, descripción)

COMPRA (dni, código, fecha_compra)

Transformación de relaciones 1:1

- Se tienen en cuenta las cardinalidades de las entidades que participan. Existen dos soluciones:

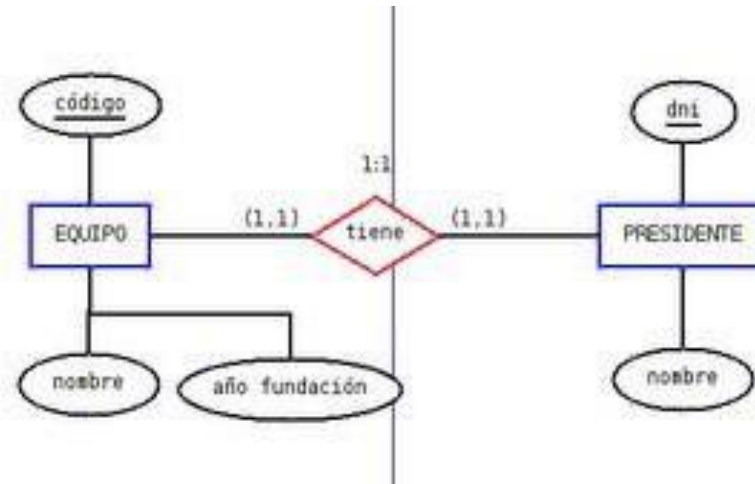
1.- Propagar la clave (regla general). Esta regla es la habitual y siempre se puede aplicar. Sin embargo, infringe la restricción de integridad referencial cuando las cardinalidades son (0,1) y (0,1). Además habría que especificar que la clave ajena admite valores nulos.

Si una de las entidades posee cardinalidad (0,1) y la otra (1,1) conviene propagar la clave de la entidad con cardinalidad (1,1) a la tabla resultante de la entidad de cardinalidad (0,1). Si ambas entidades poseen cardinalidades (1,1) se puede propagar la clave de cualquiera de ellas a la tabla resultante de la otra.

2.- Transformarla en una tabla. Si ambas entidades poseen cardinalidades (0,1) la relación se convierte en una tabla y se elegirá entre UNA de los dos AIP como clave primaria.

Transformación de relaciones 1:1

Ejemplo 1: Propagar la clave



- Solución 1:

EQUIPO (código, nombre, año_fundación)

PRESIDENTE (dni, nombre, código_equipo (FK))

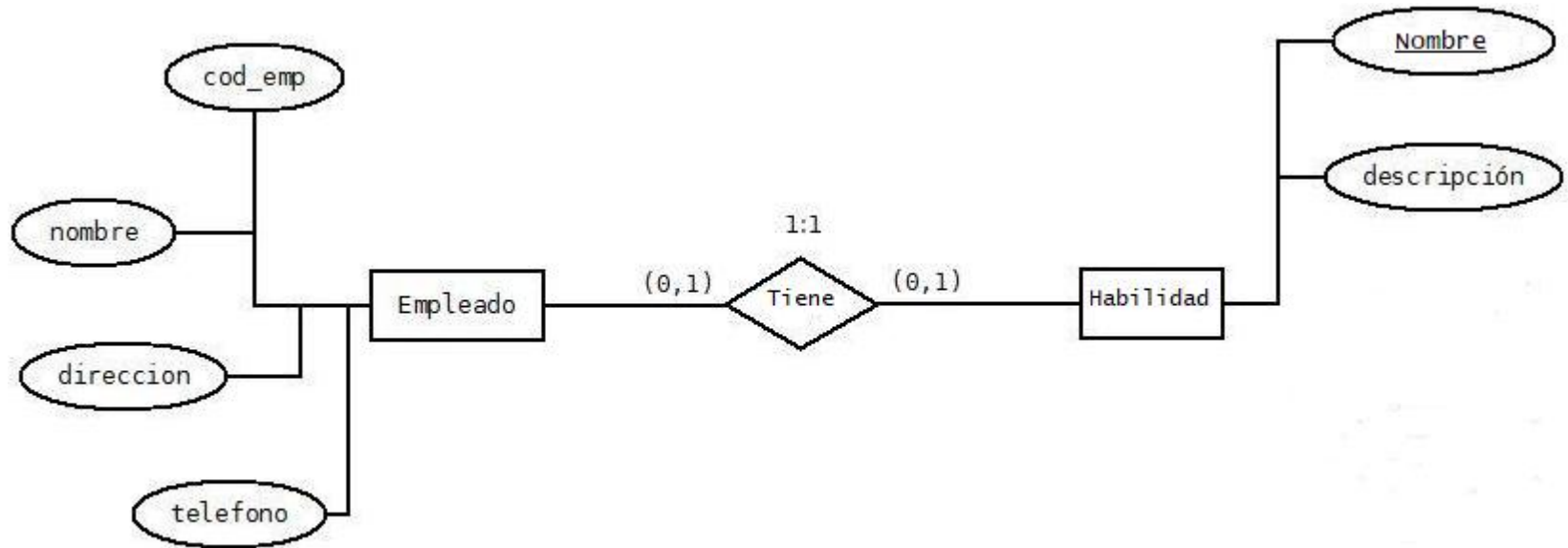
- Solución 2:

EQUIPO (código, nombre, año_fundación, dni_presidente (FK))

PRESIDENTE (dni, nombre)

Transformación de relaciones 1:1

Ejemplo 2: Transformar en una tabla



EMPLEADO (cod_empleado, nombre, dirección, teléfono)

HABILIDAD (nombre_hab, descripción)

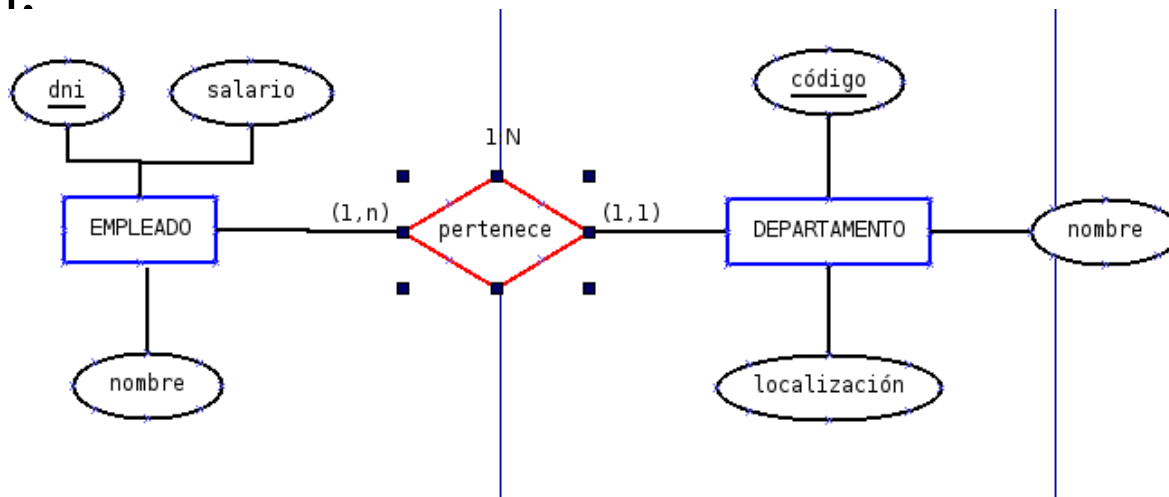
TIENE (cod_empleado, nombre_hab (FK))

Transformación de relaciones 1:N

- Existen dos soluciones:
- **Propagación de la clave (regla general).** Se propaga los AIP de la entidad que tiene la cardinalidad máxima 1 hacia la que tiene N. Esta regla es la habitual y siempre se puede aplicar. Sin embargo, infringe la restricción de integridad referencial cuando las cardinalidades son (0,1) y (x,n). Además habría que especificar que la clave ajena admite valores nulos.
- **Transformarla en una tabla.** Se hace como si se tratara de una relación N:M. Esta solución se realiza cuando se prevé que en un futuro la relación se convertirá en N:M y cuando la relación tiene atributos propios y no se quieren propagar. También se aplica cuando tenemos cardinalidad (0,1) y (x,n). La clave de esta nueva tabla es la clave de la entidad del lado muchos.

Transformación de relaciones 1:N

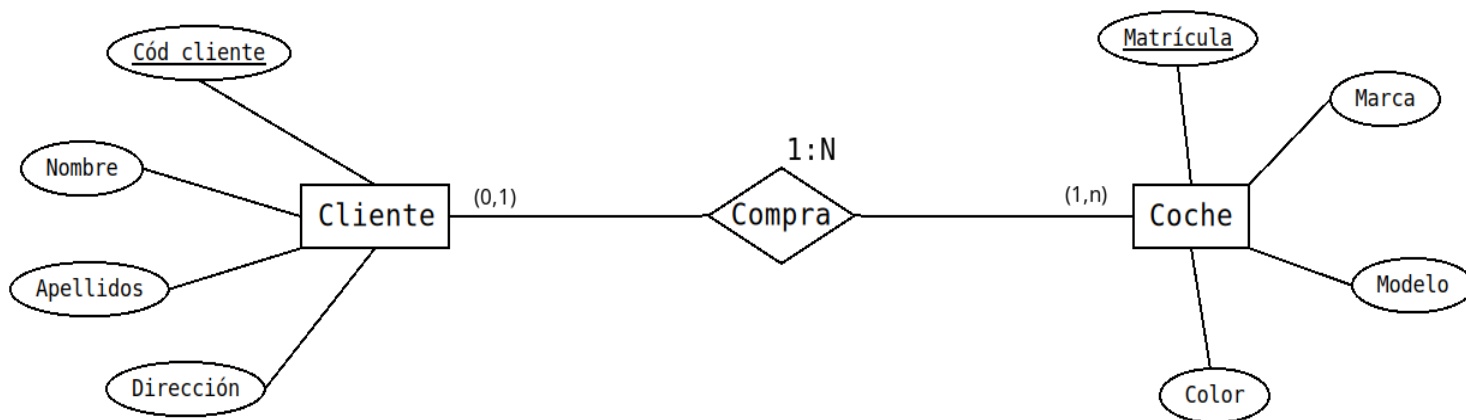
Ejemplo 1- Propagar la clave. Este caso se aplica cuando la entidad del lado 1 es obligatoria, es decir, cuando tenemos cardinalidad (1,1) y (1,n). Se propaga el atributo principal de la entidad que tiene de cardinalidad máxima 1 a la que tiene la cardinalidad máxima N, desapareciendo el nombre de la relación.



EMPLEADO (dni, nombre, salario, código_dep)
DEPARTAMENTO (código, nombre, localización)

Transformación de relaciones 1:N

Ejemplo 2 - Transformarla en una tabla. Este caso se aplica cuando la entidad del lado 1 es opcional, es decir, cuando tenemos cardinalidad (0,1) y (x,n). Se procede como si se tratara de una relación N:M, pero la clave de la nueva tabla será la clave de la entidad del lado muchos.



CLIENTE (Cód_cliente, nombre, apellidos, dirección)

COCHE (Matrícula, marca, modelo, color)

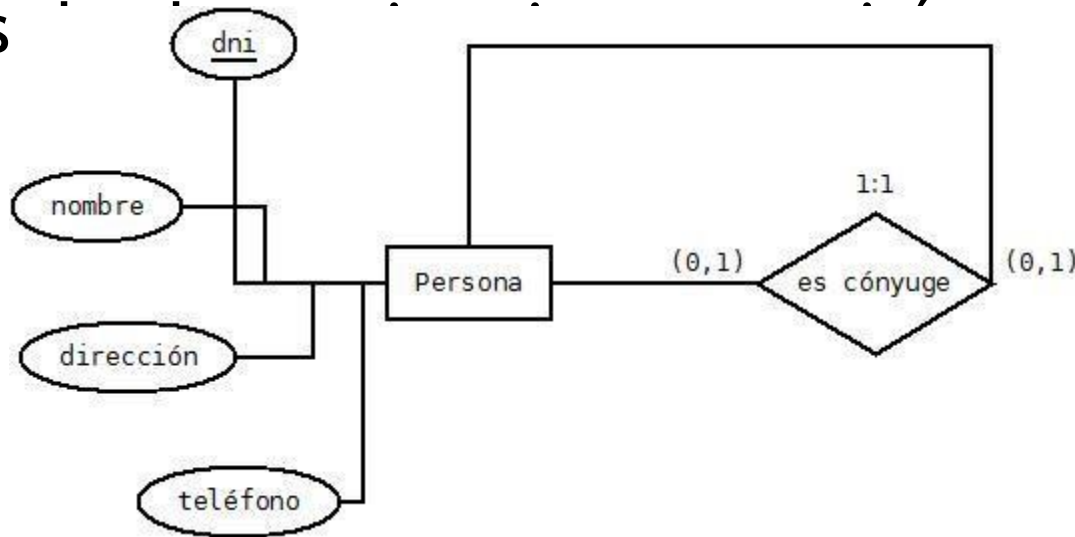
COMPRA(Matrícula, cód_cliente (FK))

Transformación de relaciones reflexivas o recursivas

- **Relación 1:1.** La clave de la entidad se repite, con lo que la tabla resultante tendrá dos veces ese atributo, una como clave primaria y otra como clave ajena de la misma.
- **Relación 1:N.** Tenemos dos casos:
 - Aquel en el que la entidad del lado 1 es obligatoria, se procede como en el caso (1:1).
 - Si no es obligatoria **y NO queremos que la clave ajena reciba valores NULL**, se crea una nueva tabla cuya clave será la de la entidad de lado N y además se repite esta clave como clave ajena.
- **Relación N:M.** Se trata igual que las relaciones binarias. Es decir, se crea una tabla nueva.

Transformación de relaciones reflexivas o recursivas (Ejemplo: Relación 1:1)

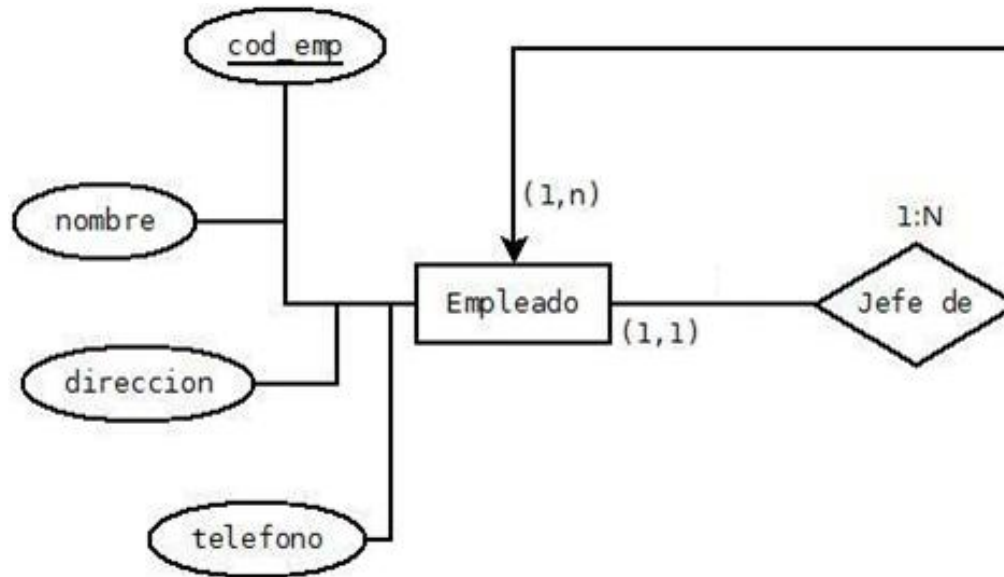
Imaginemos una tabla de PERSONAS. Tenemos una relación “es cónyuge”. Es una relación 1:1. Cada fila contendrá (por ejemplo) el DNI y todos los datos de una persona, siendo el DNI la clave primaria, y además



PERSONAS (dni, nombre, dirección, teléfono, dni_conyuge)

Transformación de relaciones reflexivas o recursivas (Ejemplo: Relación 1:N)

La relación 'Jefe', es una relación 1:N y es obligatoria. Por tanto, la Tabla EMPLEADOS queda con un atributo más, que es el número de empleado del jefe del empleado. Le ponemos como nombre JEFE.

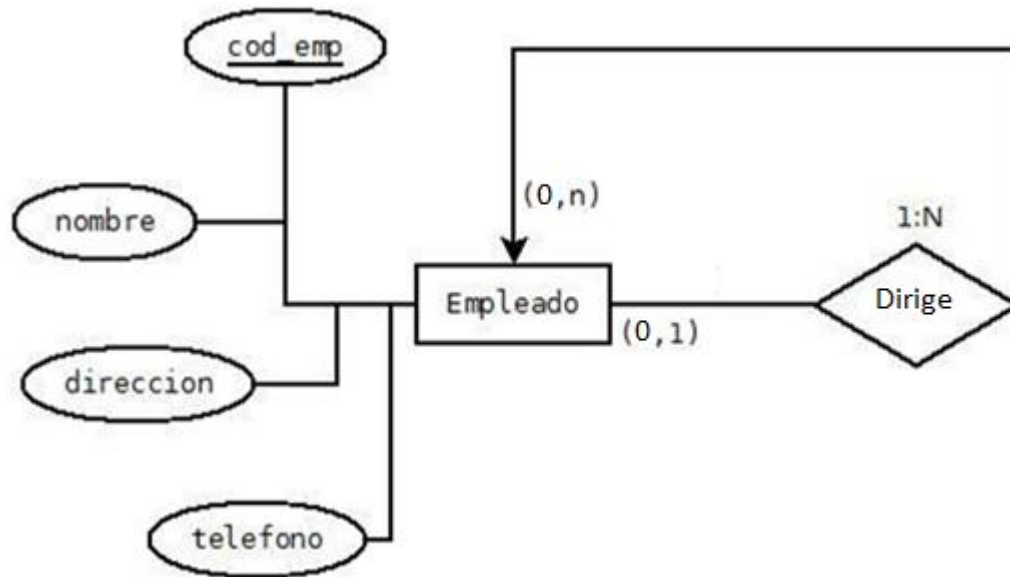


EMPLEADO (cod_emp,
jefe)

nombre, dirección, teléfono,

Transformación de relaciones reflexivas o recursivas (Ejemplo: Relación 1:N)

La relación 'Dirige', es una relación 1:N pero no es obligatoria. Por tanto, creamos una nueva tabla con clave primaria `cód_emp` y el número de empleado del jefe del empleado. Le ponemos como nombre `cód_direc`.



EMPLEADO (cod_emp, nombre, dirección, teléfono)
DIRIGE(cód_emp(FK), cód_direc(FK))

Transformación de jerarquías (generalizaciones) al modelo relacional

- El modelo relacional no dispone de mecanismo para la representación de las relaciones jerárquicas; así pues, estas relaciones se tienen que eliminar.
- Para pasar estas relaciones al modelo relacional se pueden aplicar distintas reglas.

1. Integrar todas las entidades en una única eliminando a los subtipos. Esta nueva entidad contendrá todos los atributos del supertipo, todos los de los subtipos, y los atributos discriminativos para distinguir a que subentidad pertenece cada atributo. **Se adoptara esta solución cuando los subtipos se diferencien en muy pocos atributos, y las interrelaciones sean las mismas para todos los subtipos o no existen.** Esta regla puede aplicarse a cualquier tipo de jerarquía.

- La gran ventaja es la simplicidad, pues todo se reduce a una entidad.
- El gran inconveniente es que se genera demasiados valores nulos en los atributos opcionales propios de cada entidad.

Transformación de jerarquías

(generalizaciones) al modelo relacional

2.Eliminación del supertipo. Se transfieren los atributos del supertipo a cada uno de los subtipos y las relaciones del supertipo se consideran para cada uno de los subtipos. La clave genérica del supertipo pasa a cada uno de los subtipos. Sólo puede ser aplicado para jerarquías totales y exclusivas. **Esta opción será conveniente cuando haya muchos atributos distintos en los subtipos y/o sus relaciones con otras entidades sean diferentes.**

Presenta algunos inconvenientes:

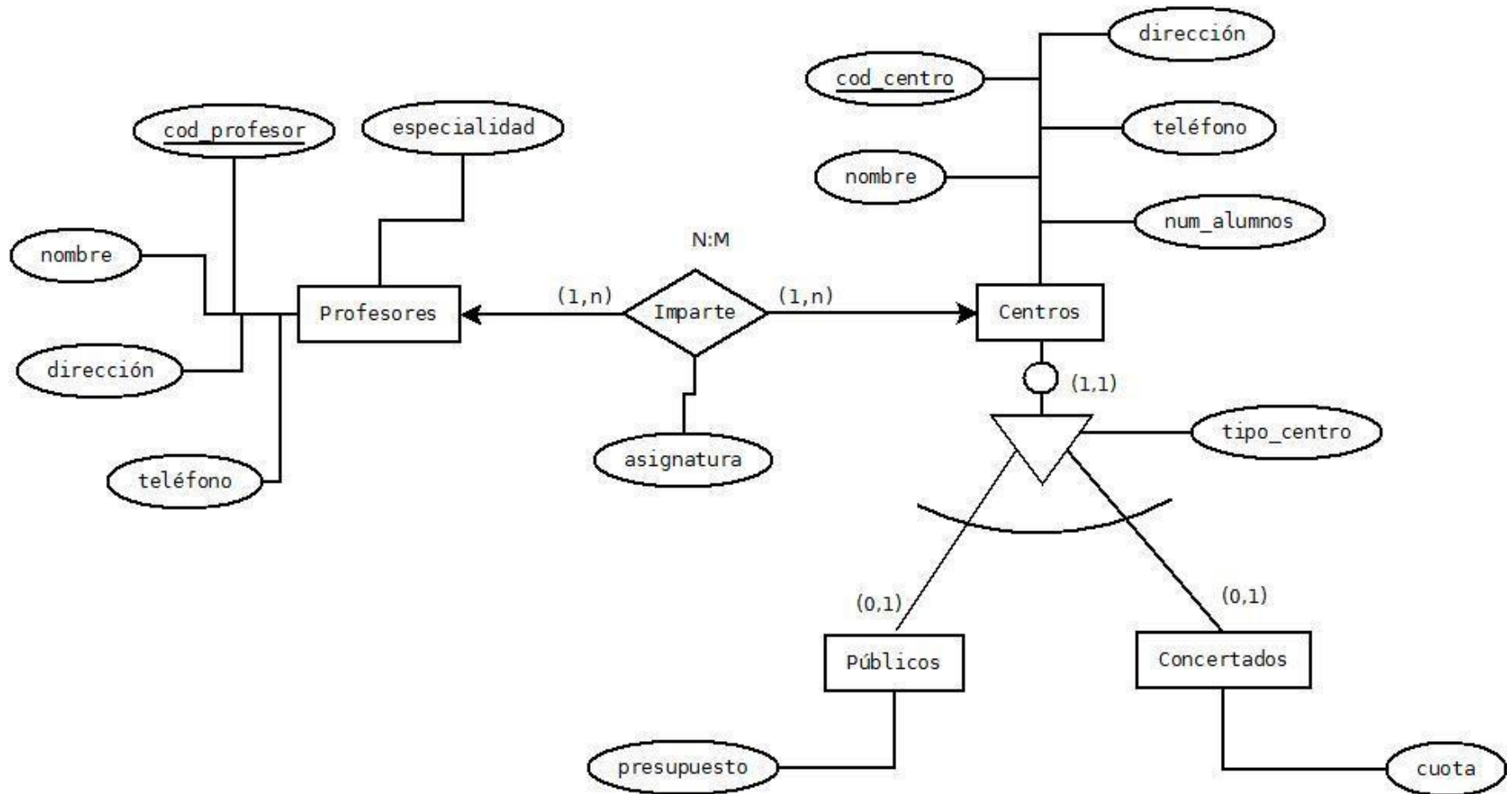
- Se introduce redundancia en la información, pues los atributos del supertipo se repiten en cada uno de los subtipos.
- El número de relaciones aumenta, pues si el supertipo tiene relaciones, éstas pasan a cada uno de los subtipos.

Transformación de jerarquías

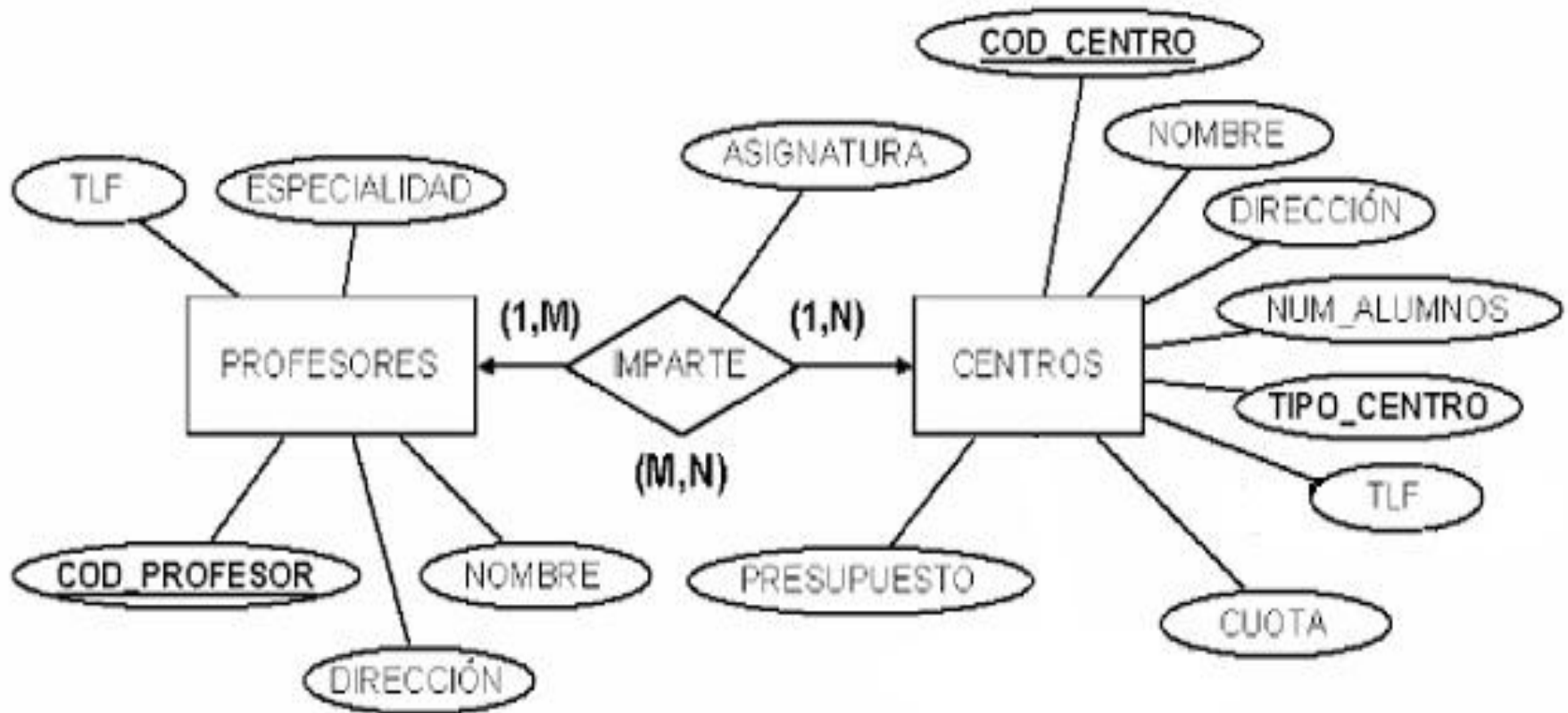
(generalizaciones) al modelo relacional

3. Insertar una relación 1:1 entre el supertipo y cada uno de los subtipos. Los atributos se mantienen y cada subtipo se identificará con la clave ajena del supertipo. El supertipo mantendrá una relación 1:1 con cada subtipo. Si la relación es exclusiva, los subtipos mantendrán la cardinalidad mínima 0, y si es solapada 0 o 1. **Esta solución, igual que el caso 2, es aconsejable cuando los subtipos tienen atributos distintos y/o sus relaciones con otras entidades sean diferentes.** Permite mantener agrupados en una tabla los atributos comunes.

Ejemplo diagrama E-R con jerarquía



Ejemplo 1: Integrar todas las entidades en una única eliminando a los subtipos.

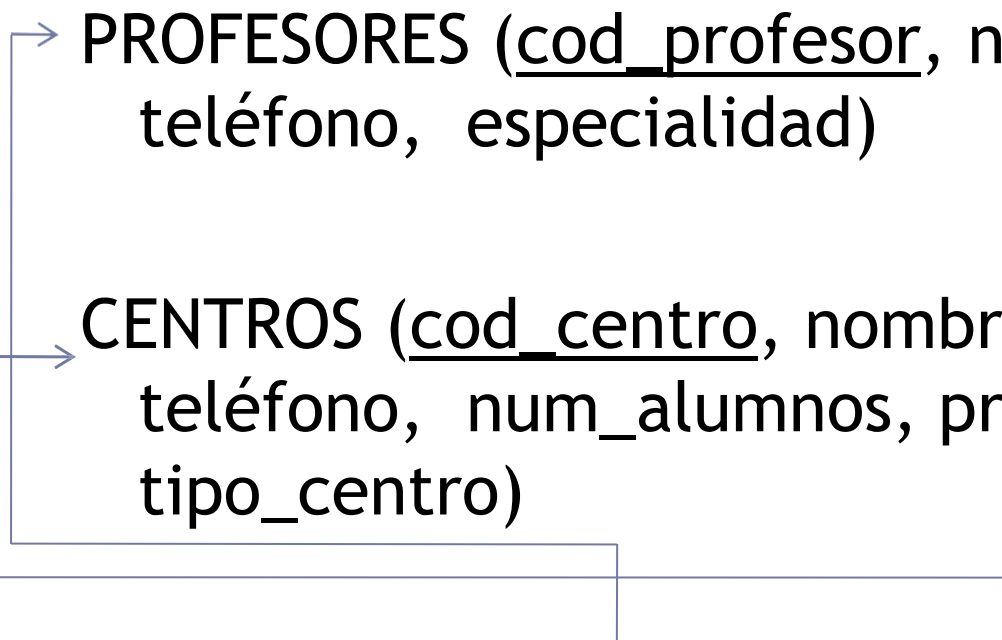


Ejemplo 1: Integrar todas las entidades en una única eliminando a los subtipos.

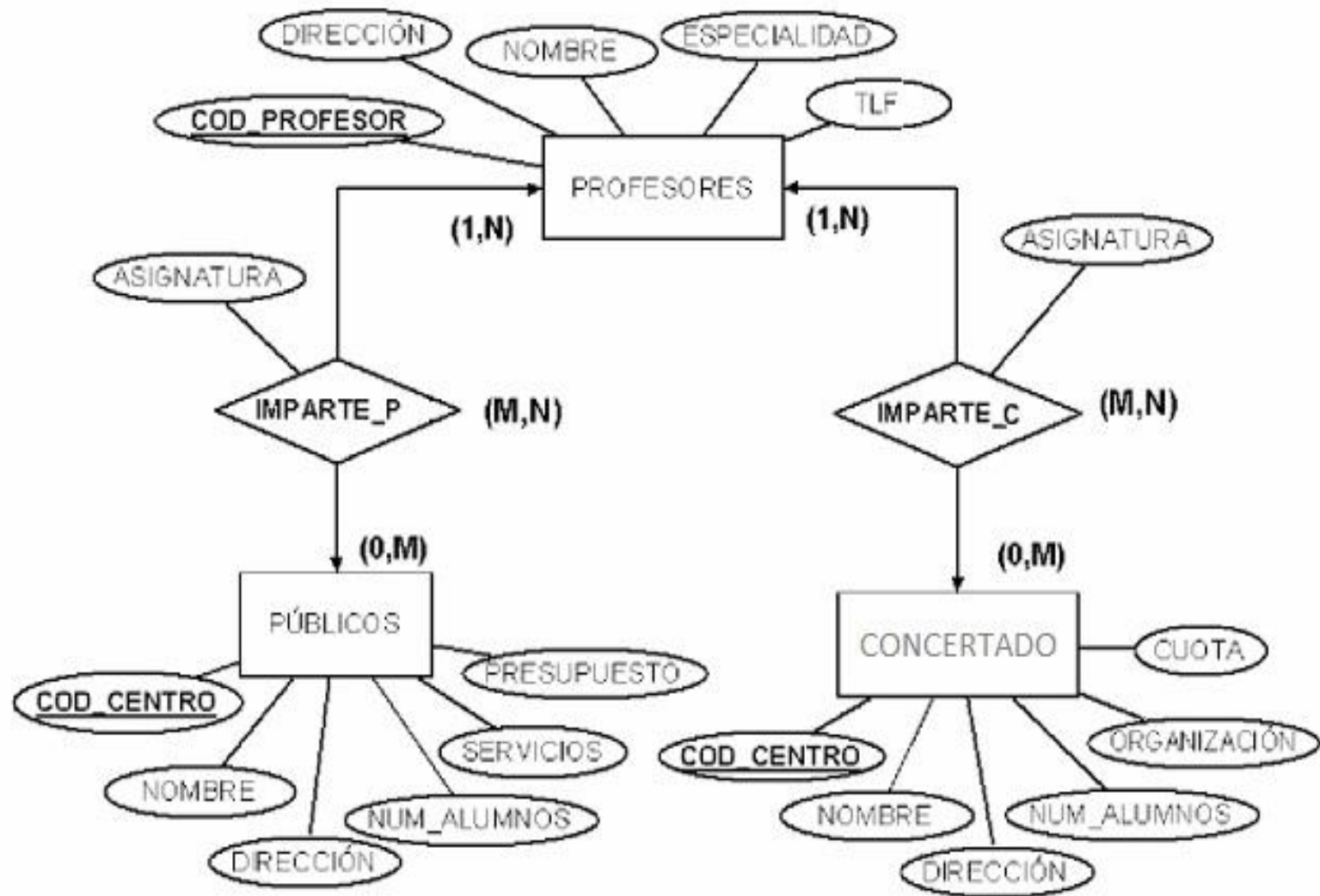
→ PROFESORES (cod_profesor, nombre, dirección, teléfono, especialidad)

→ CENTROS (cod_centro, nombre, dirección, teléfono, num_alumnos, presupuesto, cuota, tipo_centro)

IMPARTE (cod_profesor, cod_centro, asignatura)



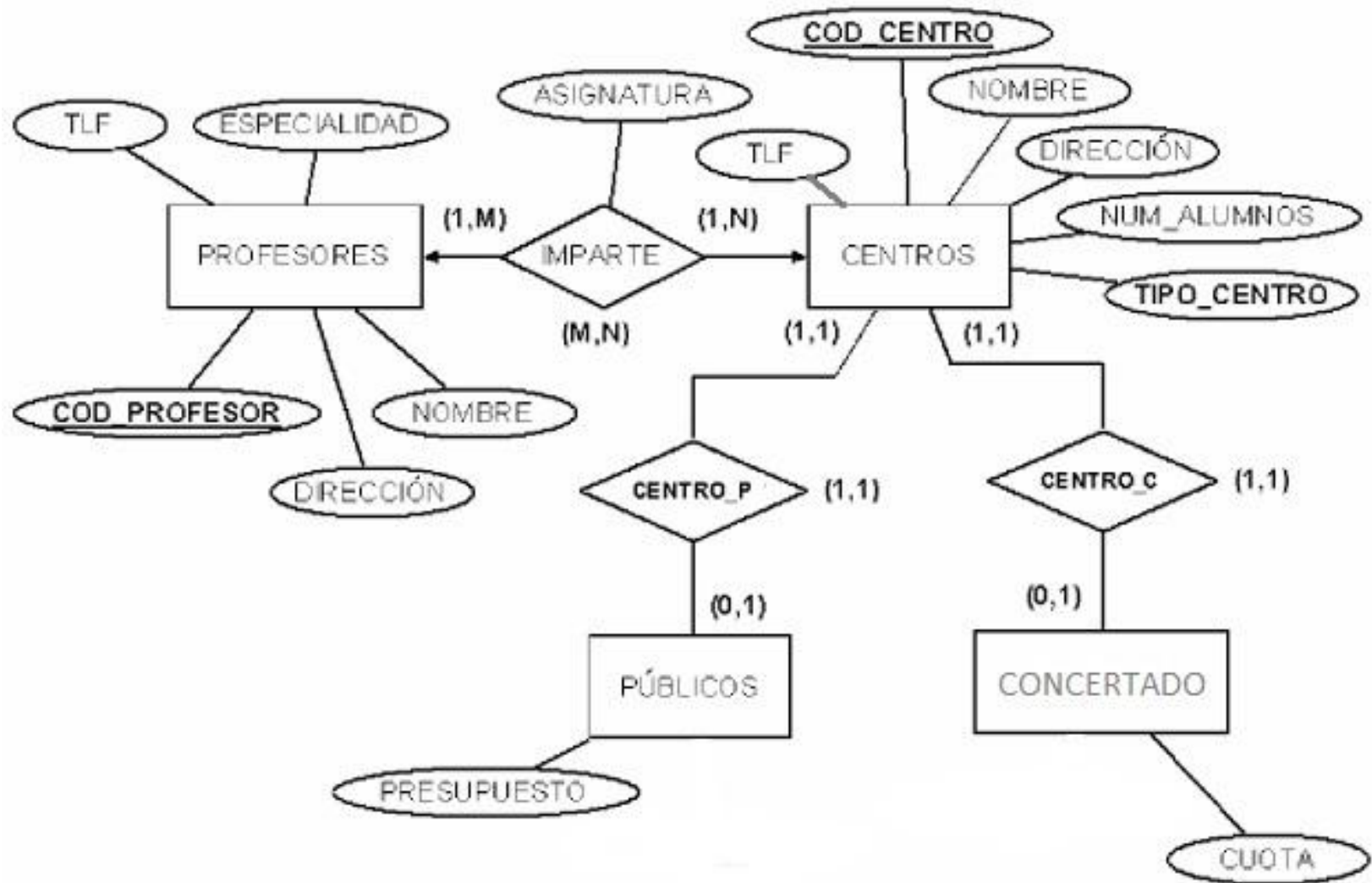
Ejemplo 2: Eliminación del supertipo.



Ejemplo 2: Eliminación del supertipo.

- PROFESORES (cod_profesor, nombre, dirección, teléfono, especialidad)
- PUBLICOS (cod_centro, nombre, dirección, teléfono, num_alumnos, presupuesto)
- CONCERTADOS (cod_centro, nombre, dirección, teléfono, num_alumnos, cuota)
- IMPARTE_P (cod_profesor, cod_centro, asignatura)
- IMPARTE _C (cod_profesor, cod_centro, asignatura)

Ejemplo 3: Insertar una relación 1:1 entre el supertipo y cada uno de los subtipos.



Ejemplo 3: Insertar una relación 1:1 entre el supertipo y cada uno de los subtipos.

→ PROFESORES (cod_profesor, nombre, dirección, teléfono, especialidad)

→ CENTROS (cod_centro, nombre, dirección, teléfono, num_alumnos, tipo_centro)

PUBLICOS (cod_centro, presupuesto)

CONCERTADOS (cod_centro, cuota)

IMPARTE (cod_profesor, cod_centro, asignatura)

Totalidad/Parcialidad

- **Totalidad**

- Prohibir las inserciones en el supertipo.
- Trigger: Al insertar un registro en el subtipo se inserte también un registro en el supertipo .
- El atributo discriminante no podrá tomar valores nulos. Tendrá que tomar el valor del subtipo insertado.

- **Parcialidad**

- Se pueden insertar registros en el supertipo (No hay que controlar nada)
- El atributo discriminante puede tomar valores nulos o un valor por defecto (Otros “O” ó Varios “V”)

Exclusividad/Inclusividad

- **Exclusividad**

- Aserción (Aserto o Trigger): Comprobar si un supertipo pertenece sólo a uno de los subtipos.
- El atributo discriminante toma valores simples (de uno de los subtipos).

- **Inclusividad o Solapamiento**

- Aserción (Aserto o Trigger): Comprobar si un supertipo pertenece a los subtipos adecuados.
- El atributo discriminante puede tomar valores combinados.

Controlar exclusividad relación jerárquica

CREATE ASSERTION Controlar_exclusividad_generalización

CHECK (Tipo="Público" AND EXIST PÚBLICO
AND NOT EXIST CONCERTADO)

OR (Tipo="Concertado" AND EXIST CONCERTADO
AND NOT EXIST PÚBLICO))

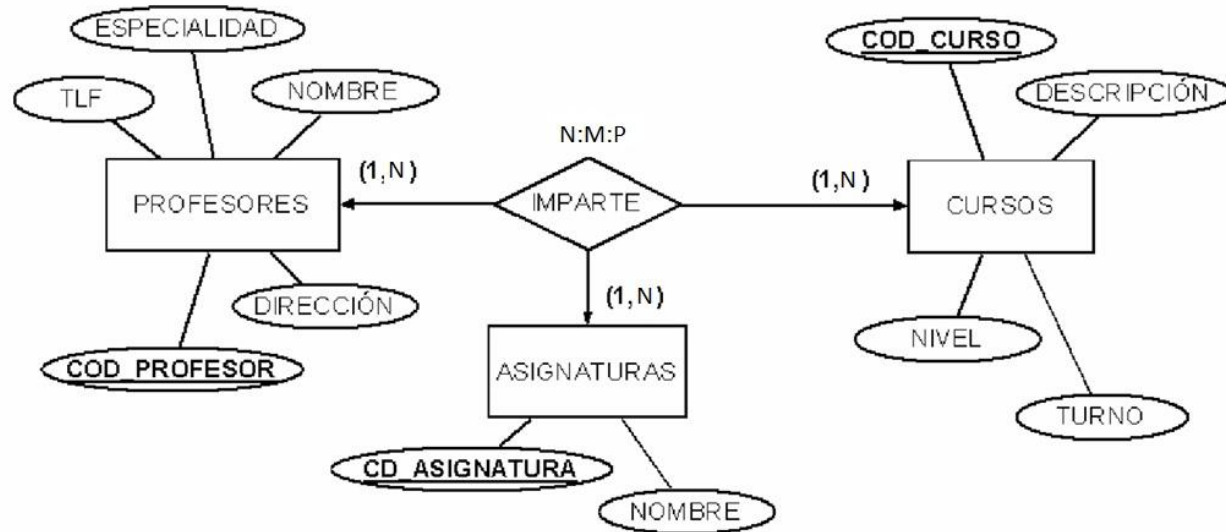
Transformación de relaciones ternarias

En este tipo de relaciones se agrupan 3 o más entidades, y para pasar al modelo de datos relacional cada entidad se convierte en tabla, así como la relación, que va a contener los atributos propios de ella más las claves de todas las entidades. La clave de la tabla resultante será la concatenación de las claves de las entidades. Hay que tener en cuenta:

- Si la relación es $N:M:P$, es decir, si todas las entidades participan con cardinalidad máxima M , la clave de la tabla resultante es la unión de las claves de las entidades que relaciona. Esa tabla incluirá los atributos de la relación si los hubiera.
- Si la relación es $1:N:M$, es decir, una de las entidades participa con cardinalidad máxima 1, la clave de esta entidad no pasa a formar parte de la clave de la tabla resultante, pero forma parte de la relación como clave ajena.
- Si la relación es $1:1:N$, es decir, dos de las entidades participan con cardinalidad máxima 1, la clave de la tabla resultante es la unión de la clave de la entidad con cardinalidad máxima y la clave de UNA de las otras dos entidades. Cualquiera de las dos.
- Si la relación es $1:1:1$, es decir, si todas las entidades participan con cardinalidad máxima 1, la clave de la tabla resultante es la unión de dos entidades cualquiera de las tres interrelacionadas.

Ejemplo de transformación ternaria N:M:P

Modelo
ER:



Modelo
Relacional:

Profesor(Cód_profesor, Nombre, Dirección, Especialidad, TLF)

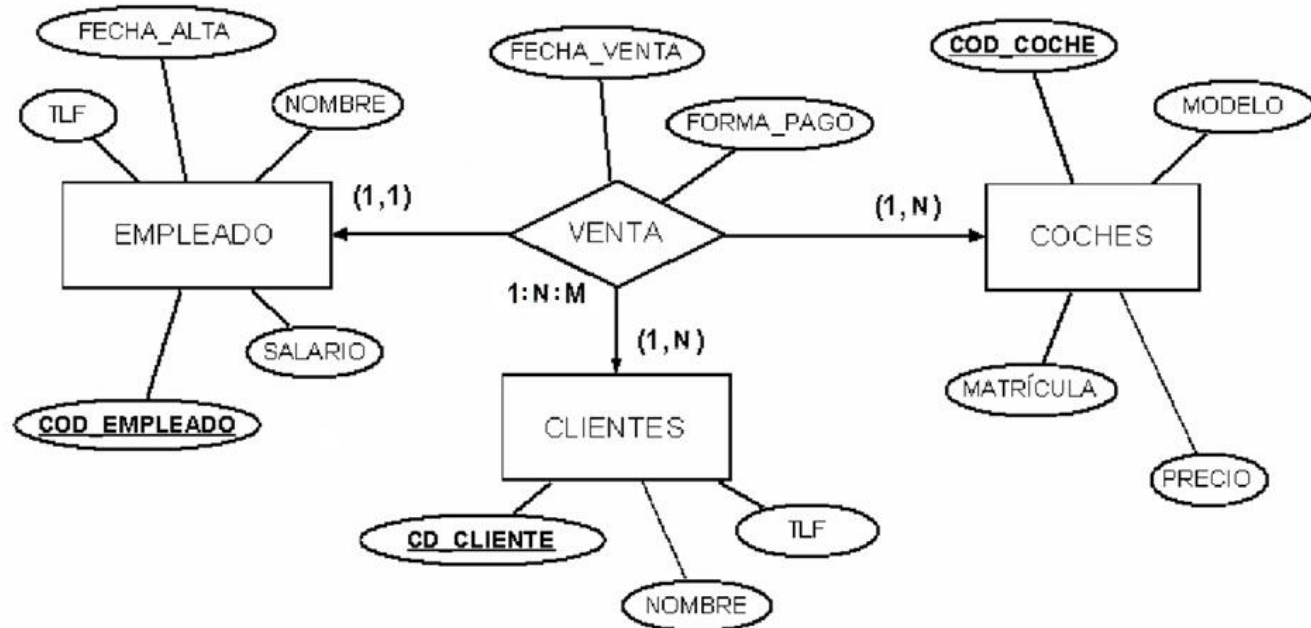
Cursos(Cód_curso, descripción, nivel, turno)

Asignatura(CD_asignatura, Nombre)

Imparte(Cód_profesor(FK), Cód_curso(FK), CD_asignatura(FK))

Ejemplo de transformación ternaria 1:N:M

Modelo
ER:



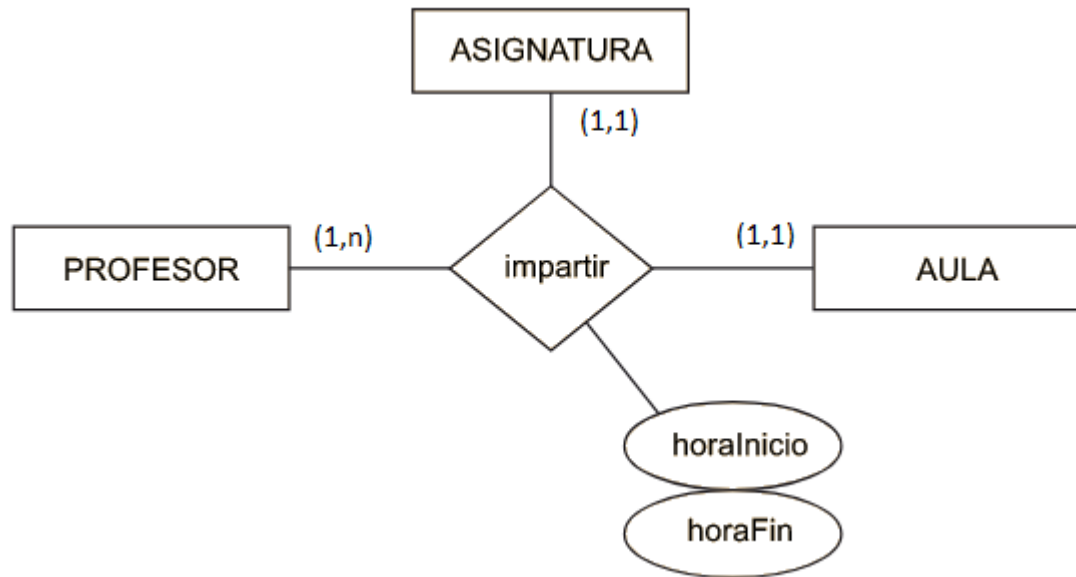
Modelo
Relacional:

Cliente(Cód_cliente, Nombre, TLF)
Empleado(Cód_empleado, Nombre, TLF, Salario, Fecha_alta)
Coche(Cód_coche, Matrícula, Modelo, Precio)
Venta(Cód_coche(FK), Cód_cliente(FK), Cód_empleado, Forma_pago, Fecha_venta)

Ejemplo de transformación ternaria 1:1:N

Modelo

ER:



Modelo

Relacional:

En este caso, encontraremos dos posibles soluciones. Si bien las relaciones que derivan de las entidades serán las mismas:

Ejemplo de transformación ternaria 1:1:N

PROFESOR(DNI, nombre, apellidos, correoElectronico)

AULA(código, aforo)

ASIGNATURA(código, nombre)

Encontramos dos posibles soluciones a la representación de la interrelación:

Solución 1:

IMPARTIR(DNI, códigoAula, códigoAsignatura, horaInicio, horaFinal)

donde {códigoAula} referencia AULA(código)

donde {códigoAsignatura} referencia ASIGNATURA(código)

donde {DNI} referencia PROFESOR

y {DNI, códigoAula} es clave primaria.

Solución 2:

IMPARTIR(DNI, códigoAsignatura, códigoAula, horaInicio, horaFinal)

donde {códigoAula} referencia AULA(código)

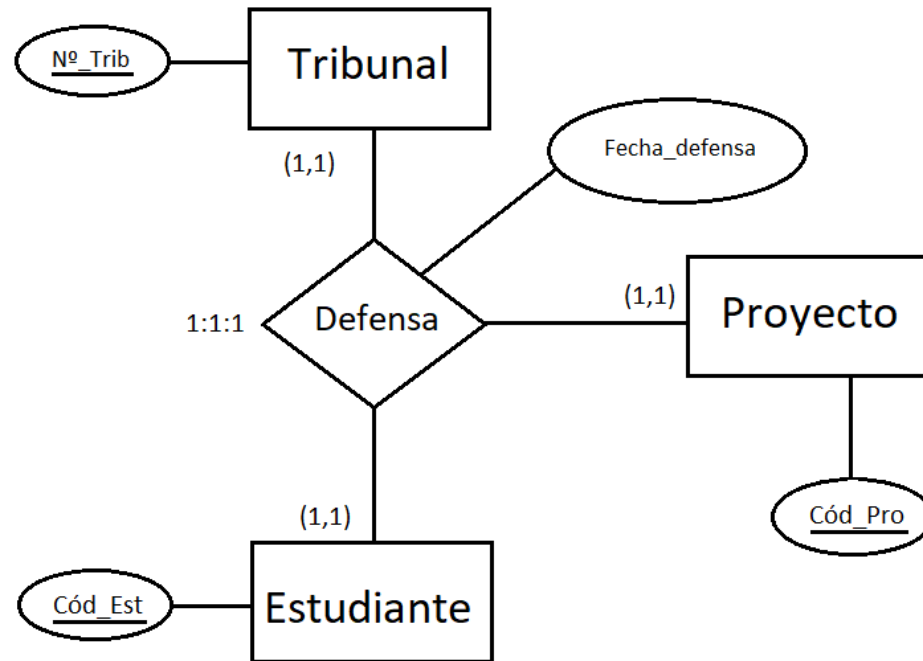
donde {códigoAsignatura} referencia ASIGNATURA(código)

donde {DNI} referencia PROFESOR

y {DNI, códigoAsignatura} es clave primaria.

Ejemplo de transformación ternaria 1:1:1

Modelo
ER:



Modelo
Relacional:

Tribunal(Nº_Trib, ...)

Estudiante(Cód_Est, ...)

Proyecto(Cód_Pro, ...)

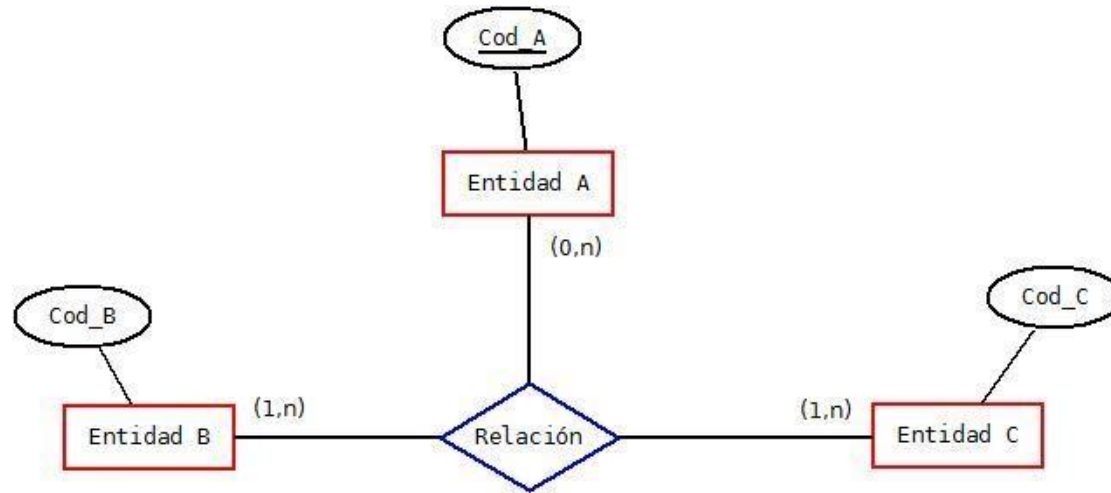
Defensa(Nº_Trib(FK), Est(FK), Cód_Pro(FK), Fecha_defensa)

Transformación de relaciones ternarias con cardinalidad mínima cero en alguna entidad.

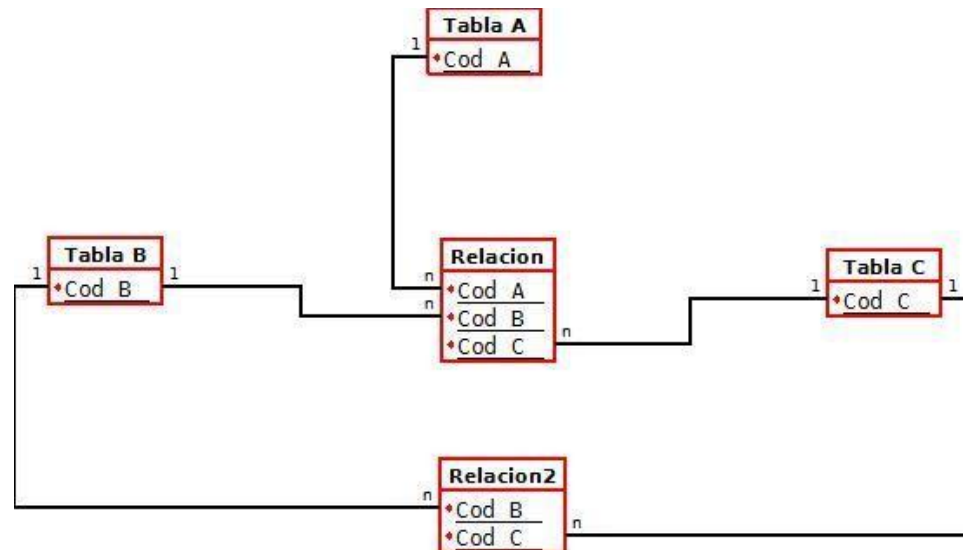
- Sí algunas de las entidades tuviera cardinalidad mínima cero **y NO queremos que la clave ajena reciba valores NULL** se crearían dos tablas para la relación.
- Sí dos de las entidades tuvieran cardinalidad mínima cero **y NO queremos que la clave ajena reciba valores NULL** se crearían tres tablas para la relación.

Ejemplo de transformación ternaria N:M:P con una entidad con cardinalidad mín. cero

Modelo
ER:



Modelo
Relacional:



Transformación de relaciones n-arias.

En todos los casos, la transformación de una interrelación n-aria consistirá en la obtención de una nueva relación que contiene todos los atributos que forman las claves primarias de las n entidades interrelacionadas y todos los atributos de la interrelación.

Podemos distinguir los casos siguientes:

- a) Si todas las entidades están conectadas con “muchos”, la clave primaria de la nueva relación estará formada por todos los atributos que forman las claves de las n entidades interrelacionadas.
- b) Si una o más entidades están conectadas con “uno”, la clave primaria de la nueva relación estará formada por las claves de $n - 1$ de las entidades interrelacionadas, con la condición de que la entidad, cuya clave no se ha incluido, debe ser una de las que está conectada con “uno”.

Resumen de transformaciones

Modelo Entidad-Relación		Modelo relacional (Transformación)
Entidad		Tabla
Atributo		Campo
Relaciones	1:1	(1,1) - (1,1) Propagación de la clave. 3 Opciones: - De la entidad A a la B - De la entidad B a la A (0,1) - (1,1) Propagación de la clave del lado (1,1) al lado (0,1) (0,1) - (0,1) Se crea una tabla que tiene por clave primaria las claves ajenas de ambas entidades
	Binarias	(1,1) - (x,n) Propagación de la clave del lado 1 al lado N. (0,1) - (x,n) Se crea una tabla que tiene por clave primaria la clave del lado N y sólo como ajena a la clave del lado 1.
	1:N	1:N Existencia: Se tratan igual de las 1:N. 1:N Identificación: Propagación de la clave del lado 1 al lado N, pasando a formar parte de la clave primaria de la entidad débil.
	N:M	Se crea una tabla que tiene por clave primaria a las claves primarias de las tablas que relaciona. Si tiene atributos propios, habrá que observar si hay que “ampliar la clave”.

Resumen de transformaciones

Modelo Entidad-Relación		Modelo relacional (Transformación)
Relaciones	Reflexivas	Igual que las binarias*
	Ternarias y n-arias	Se crea una tabla que tiene por clave primaria a las claves primarias de las tablas que relaciona. A observar: - Si tiene atributos propios, habrá que estudiar si es necesario “ampliar la clave”. - Si hay entidades con participaciones (1,1) o (0,1), habrá que estudiar si se puede “reducir la clave” quitando de la clave primaria las claves ajenas de dichas Entidades
	Jerárquicas	La superentidad crea una tabla a no ser que posea muy pocos atributos, en cuyo caso desaparecería. 1. Las subentidades crearán una tabla si y sólo si tienen atributos propios o bien se relacionan con otras entidades del modelo. 2. Las subentidades heredan la clave primaria de la superentidad. 3. En el caso de tener una jerarquía: a. Exclusiva: el atributo ‘tipo’ se sube a la superentidad y se le asigna una codificación que identifique a cada una de las subentidades. b. Inclusiva: se crea una tabla que almacene las relaciones entre la superentidad y las subentidades de la siguiente forma: es_un (#clave_superentidad, #tipo)